

第 3 章 栈和队列

郑冠杰

2026 年 3 月 16 日

有一个国家的外交信件使用如下方法写成密码（密码由 26 个小写字母、空格、标点符号组成）：首先颠倒所有的非元音字母连成的字段序列（包括空格和标点符号，其中元音字母为 a,e,i,o,u），然后再把全部信件倒着写。请你编写一个解密程序翻译信件。

- 例如：

原文：

data structures

先颠倒所有的非元音字段（包括空格和标点符号）：

datarts utcures

再全部倒着写（最终密码）：

seructu stratad

- ▶ 栈

- ▶ 顺序栈
- ▶ 链式栈
- ▶ 栈的应用

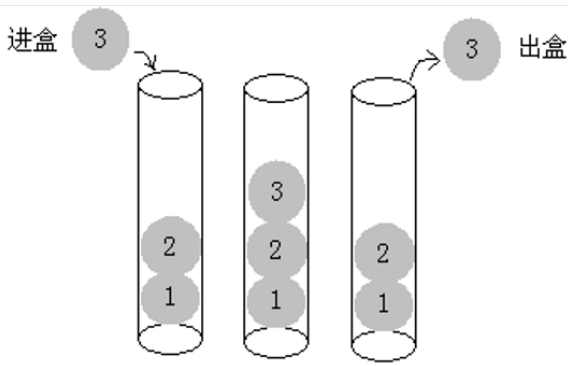
- ▶ 队列
- ▶ 顺序循环队列
- ▶ 链式队列
- ▶ 优先队列
- ▶ 队列的应用

栈的定义

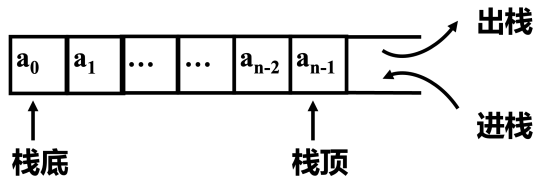
- ▶ 如果元素到达线性结构的时间越晚，离开的时间就越早，这种线性结构称为**栈 (Stack) 或堆栈**；
- ▶ 因为元素之间的关系是由到达、离开的时间决定的，因此栈通常被称为**时间有序表**。
- ▶ 而到达和离开的含义就是插入和删除操作，因此栈可以看作是插入和删除操作位置受限的线性表。

栈的定义

- ▶ 乒乓球盒的进球和出球，它遵循了最后进盒的球反而最先出盒，即所谓的后进先出 (LIFO, Last In First Out) 或先进后出 (FILO, First In Last Out) 结构。

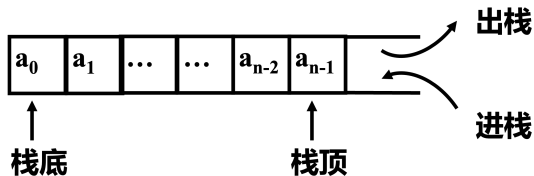


栈相关术语



- ▶ 栈的首部（元素最早到达的部分）称为栈底（bottom），栈结构的尾部（元素最晚到达的部分）称为栈顶（top）。
- ▶ 为了保证栈的先进后出或后进先出的特点，元素的插入和删除操作都必须在栈顶进行。元素从栈顶删除的行为，称为出栈或者弹栈操作（pop）；元素在栈顶位置插入的行为，称为进栈或者压栈操作（push）。

栈相关术语



- ▶ 读取栈顶元素数据值的操作，称为取栈顶内容操作 (top)。
- ▶ 当栈中元素个数为零时，称为空栈。

栈的抽象数据类型

Data: $\{x_i | x_i \in ElemSet, i = 1, 2, 3, \dots, n, n > 0\}$ 或 Φ ; ElemSet 为元素集合。

Relation: $\{< x_i, x_{i+1} > | x_i, x_{i+1} \in ElemSet, i = 1, 2, 3, \dots, n - 1\}$, x_1 为栈底, x_n 为栈顶。

Operations:

initialize 输入: 无 输出: 栈初始化为一个空栈。

isEmpty 输入: 无 输出: 栈 Stack 空返回 true, 否则返回 false。

isFull 输入: 无 输出: 栈 Stack 满返回 true, 否则返回 false。

top 前提: 栈 Stack 非空。 输出: 返回栈顶元素的值, 栈顶元素不变。

push 前提: 栈 Stack 非满, 已知待插入的元素。 输出: 将该元素压栈, 使其成为新的栈顶元素。

pop 前提: 栈 Stack 非空。 输出: 将栈顶元素弹栈, 该元素不再成为栈顶元素。

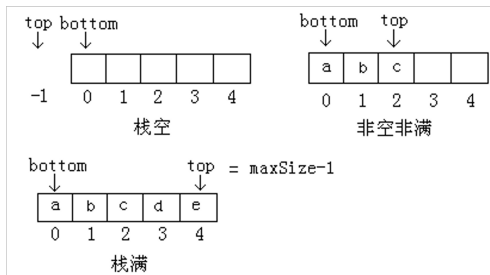
destroy 输入: 无 结果: 释放栈 Stack 占用的所有空间。

栈和队列

- ▶ 栈
 - ▶ 顺序栈
 - ▶ 链式栈
 - ▶ 栈的应用
- ▶ 队列
 - ▶ 顺序循环队列
 - ▶ 链式队列
 - ▶ 优先队列
 - ▶ 队列的应用

顺序栈

- 栈的顺序存储即使用连续的空间存储栈中的元素。可以将栈底放在数组的 0 下标位置，进栈和出栈总是在栈的同一端（栈顶）进行，顺序方式存储的栈称为顺序栈。



`bottom`总是0，不需要记忆

栈空标志: `top = -1`

栈满标志: `top = maxSize - 1`

顺序栈类的声明

顺序栈的描述：数组指针 array，数组大小 maxSize，栈顶下标 Top。

seqStack

```
class illegalSize{};
class outOfBound{};

template <class elemType>
class seqStack
{private:
    elemType *array; //栈存储数组，
                     //存放实际的数据元素。
    int Top; //栈顶下标。
    int maxSize; //栈中最多能存放的
                 //元素个数。
    void doubleSpace();
```

```
public:
    seqStack(int initSize = 100); //初始化顺序栈
    bool isEmpty () { return ( Top == -1 ); } ; //栈空
                                     //返回 true, 否则返回 false。
    bool isFull () { return (Top == maxSize-1); }; //栈
                                     //满返回 true, 否则返回 false。
    elemType top (); // 返回栈顶元素的值，不改变栈顶
    void push (const elemType &e ); //将元素e压入栈顶，
                                     //使其成为新的栈顶。
    void pop (); //将栈顶元素弹栈。
    ~seqStack(){ delete []array; }; //释放栈占用的动态
                                     //数组
};
```

顺序栈类成员函数的实现

seqStack

```
template <class elemType>
seqStack<elemType>::seqStack(int
    initSize)//初始化顺序栈
{
    array = new elemType[initSize];
    if (!array) throw illegalSize();
    Top=-1;    maxSize=initSize;
}

template <class elemType>
elemType seqStack<elemType>::top ()// 返回栈顶元素的值，不改变栈顶
{
    if (isEmpty()) throw outOfBound();
    return array[Top];
}
```

```
template <class elemType>
void seqStack<elemType>::push(const elemType &e )
//将元素e压入栈顶，使其成为新的栈顶元素。
{
    if (isFull()) doubleSpace();
    //栈满时从新分配2倍的空间，并将原空间内容拷贝入
    array[++Top] = e; // 新结点放入新的栈顶位置。
}

template <class elemType>
void seqStack<elemType>::pop()//将栈顶元素弹栈。
{
    if (Top== -1) throw outOfBound();
    Top--;
}
```

基本操作效率分析

函数 `initialize(seqStack)`、`isEmpty`、`isFull`、`top`、`pop`、`destroy ( seqStack)` 的时间复杂度均为 $O(1)$ 。

`push` 因某时可能扩大空间，造成 $O(n)$ 时间消耗，但按照“分期付款式”法，分摊到单次的插入操作，时间复杂度仍为 $O(1)$ 。——**均摊法**

顺序栈的应用（测试）

编写程序从键盘上依次输入一串字符（以回车键结束）。要求将该串字符按照输入顺序的逆序在屏幕上输出。

在程序中可以建立一个顺序栈，将输入的字符依次入栈，最后再依次出栈，便能得到逆序结果。

顺序栈结构的应用 (main.cpp)

seqStack

```
#include <iostream>
#include "seqStack.h"
using namespace std;
int main()
{    // 声明一个栈。
    seqStack<char> s;
    char ctemp;
    //从键盘输入若干字符（结束用回车），
    //依照输入次序分别进栈
    cout<<"Input the elements, "
         <<"press enter to an end: ";
    ctemp = cin.get();
    while ( ctemp!='\n')
    {    s.push(ctemp);    ctemp = cin.get();    }
```

```
//将栈中的结点逐个出栈，并输出到
//屏幕上。
    cout<<"output the elements in
         the stack one by one:";
    while ( !s.isEmpty() ) {
        ctemp = s.top();
        s.pop();
        cout<<ctemp;
    }
    cout<<endl;
    return 0; }
```

共享栈

在实际应用中，有时需要同时使用多个数据类型相同的栈。

栈中的元素个数因进栈、出栈操作动态地变化，所有栈不一定同时达到栈满，有时一些栈满而另一些栈尚余空间。

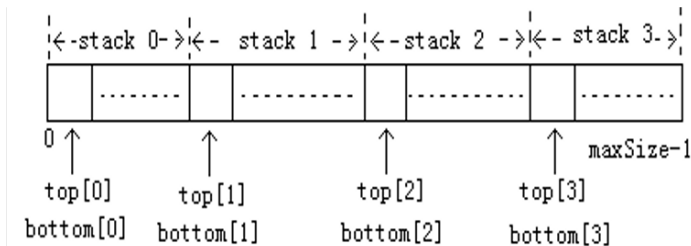
为了提高空间使用率，可以在同一块连续的空间中设置多个栈。

多个栈间共享空间，这些栈称为“共享栈”。

共享栈的特点是每个栈拥有一个连续的小空间，所有共享栈拥有一个大的连续空间。

共享栈

top 指向实际栈顶的后一个位置



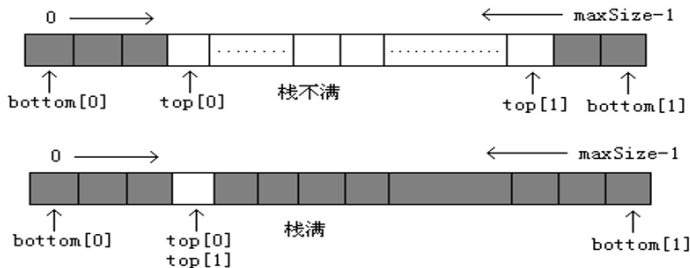
假设有 m 个栈，第 i 个栈空的条件： $\text{top}[i] = \text{bottom}[i]$ ；

第 i 个栈栈满条件为：当 $i < m-1$ 时， $\text{top}[i] = \text{bottom}[i+1]$ ；

当 $i = m-1$ 时， $\text{top}[i] = \text{maxSize}$ 。

双共享栈

可以将两个栈相向设置，即两个栈的栈底分别设置在连续空间的两个端点。



栈空的条件 $\text{top}[i] = \text{bottom}[i]$, $i=0$ 或 1 , 两个栈不一定同时为空;

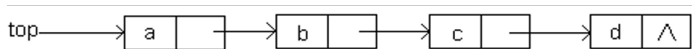
栈满的条件 $\text{top}[0] = \text{top}[1]$, 即两个栈当中只剩下一个空位置的时候栈满, 两个栈必定同时栈满。

栈和队列

- ▶ 栈
- ▶ 顺序栈
- ▶ 链式栈
- ▶ 栈的应用
- ▶ 队列
- ▶ 顺序循环队列
- ▶ 链式队列
- ▶ 优先队列
- ▶ 队列的应用

顺序栈

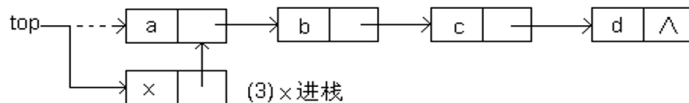
- ▶ 用不连续的空间和附加指针来存储元素及元素间的关系。
- ▶ 栈顶指针 top 指向处于栈顶的结点，即单链表中的首结点。



(1) 链式栈



(2) a 出栈



(3) x 进栈



(4) 空栈

问题：为啥没有头结点了？

链式栈基本操作分析

进栈操作 push 的实现，按照以下操作顺序：

- 1) 申请新的结点空间，data 字段为进栈元素值，next 字段指向首结点。
- 2) 栈顶指向新结点。

出栈操作 pop 的实现，按照以下操作顺序：

- 1) 记住栈顶结点的地址。
- 2) 将原栈顶的直接后继设为新的栈顶。
- 3) 释放原来栈顶结点空间。

链式栈类的声明

linkStack

```
class outOfBound{};
template <class elemType>
class linkStack;
template <class elemType>
class Node
{ friend class linkStack<elemType>;
  private:
    elemType data;
    Node *next;
  public:
    Node(){next = NULL;}
    Node(const elemType &x, Node
          *p=NULL)
    { data = x; next = p; }
};
```

```
template <class elemType>
class linkStack
{
  private:
    Node<elemType> *Top;
  public:
    linkStack(){ Top = NULL; }; //初始化栈，使其为
    空栈
    bool isEmpty(){ return (Top==NULL); }; //栈为
    空返回true，否则返回false。
    bool isFull(){ return false; }; //栈满true，否
    则false。结点空间不连续，故总能满足
    elemType top();
    void push(const elemType &e);
    void pop();
    ~linkStack();
};
```

链式栈类的声明

linkStack

```
template <class elemType>
elemType linkStack<elemType>::
    top()
{
    if (!Top) throw outOfBound
        (); //栈空
    return Top->data;
}

template <class elemType>
void linkStack<elemType>::push(
    const elemType &e)
{ Top = new Node<elemType>(e,
    Top); }
```

```
template <class elemType>
void linkStack<elemType>::pop()
{
    Node<elemType> *tmp;
    if (!Top) throw outOfBound
        (); //栈空

    tmp = Top; //用tmp记住原栈
               顶结点空间，用于弹栈后
               的空间释放
    Top = Top->next; //实际将栈
                   顶结点弹出栈

    delete tmp; //释放原栈顶结点
               空间
}
```

```
template <class elemType>
linkStack<elemType>::~~
    linkStack()
{
    Node<elemType> *tmp;
    while (Top)
    {
        tmp = Top;
        Top = Top->next;
        delete tmp;
    }
}
```

链式栈性能分析

- ▶ 析构函数将栈中的所有结点清除，空间回收，时间复杂度为 $O(n)$ 。
- ▶ 构造函数、isEmpty、isFull、top、push、pop 的时间复杂度均为 $O(1)$ 。

栈和队列

- ▶ 栈
- ▶ 顺序栈
- ▶ 链式栈
- ▶ 栈的应用
- ▶ 队列
- ▶ 顺序循环队列
- ▶ 链式队列
- ▶ 优先队列
- ▶ 队列的应用

- ▶ 函数调用
- ▶ 括号匹配
- ▶ 表达式计算

函数的调用和返回
递归示例

栈的应用——递归函数的非递归实现

- 当一个函数在调用另一个函数时，要做三件事：
 1. 保存参数，返回地址等信息，**保护现场**；
 2. 为**被调用函数**的局部变量**分配存储区**；
 3. 将**控制转移**到被调用函数的入口。
- 从被调用函数返回调用函数之前，也要做三件事：
 1. 保存被调用函数的计算结果，**恢复现场**；
 2. **释放被调函数**的数据区；
 3. 将**控制转回**到调用函数。

栈的应用——递归函数的非递归实现

- 函数调用时通过栈实现的；
- 递归本质是函数调用；
- 递归可以用栈消除。

函数执行过程

Main

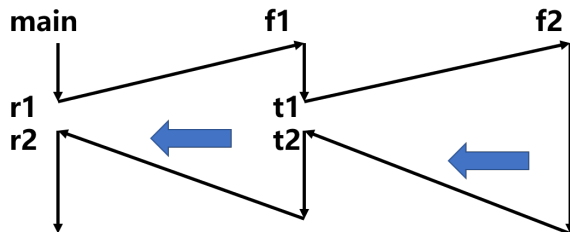
```
void main()
{
    .....
    r1: f1();
    r2:
    .....
}
```

F1

```
void f1()
{
    .....
    t1: f2();
    t2:
    .....
}
```

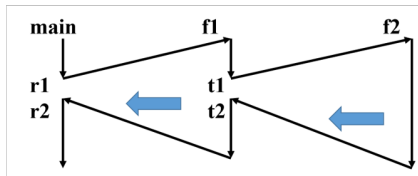
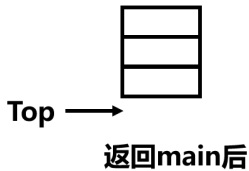
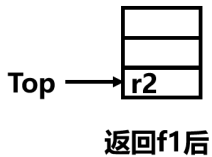
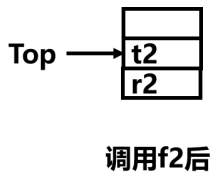
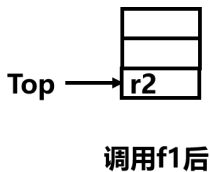
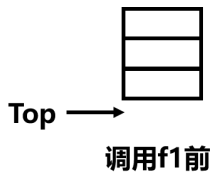
F2

```
void f2()
{
    .....
    .....
    .....
}
```



函数调用的实现

- 设置一个栈模拟函数调用。当发生函数调用时，将**当前的函数的现场进栈**。



- 递归是一种特殊的函数调用，是在一个函数中又调用了**函数本身**。
- **递归程序的本质是函数调用**，而函数调用是要花费额外的时间和空间（如**函数副本**、**栈**）。
- 在系统内部，函数调用是用栈来实现，如果程序员可以自己**控制**这个栈，就可以消除递归调用。

递归例子：求 N!

F1

```
int f1(int n)
{
    if (n < 0) return 0;
    if ((n == 0) || (n == 1))
        return 1;
    return n * f1(n-1);
}
```

F2

```
int f2(int n)
{
    seqStack<int> s;
    int m, sum;
    sum = 1;
    while (n != 0)
    {s.push(n); n--;}
    while (!s.isEmpty())
    {m = s.pop(); sum = sum * m;}
    return sum;
}
```

1

2

3

4

- 看清递归时的计算路径

栈的应用

- ▶ 函数调用
- ▶ 括号匹配
- ▶ 表达式计算

括号配对

- ▶ 编译器做语法检查的任务之一是检查符号是否配对，最简单的符号匹配问题是括号是否匹配，如开括号（及后面必须依次跟随相应的闭括号及）。
- ▶ 如下段程序中的括号、引号是否匹配。

```
int main()
{
    int a[20], i;
    for (i=0; i<20; i++)
    {
        a[i]=3*(19-i)/5*(12-6);
        cout<<a[i]<< '/t' ;
    }
    return 0;
}
```

表达式用栈检查示例

括号匹配算法

- ① 首先创建一个字符栈。
- ② 从源程序中读入字符。
- ③ 如果字符为结束符，转向 5。
 - 如果读入的是开括号，将其进栈。
 - 如果读入的是闭括号但栈是空的，说明少开括号，报错并结束。
 - 如果读入的是闭括号但栈不空，将栈中的开括号出栈。如果出栈的开括号和读入的闭括号不是同种类型（如一个为小括号，一个为中括号），说明不匹配，报错并结束。
- ④ 继续从源程序中读入下一个符号，转向 3。
- ⑤ 如果栈非空，说明开括号多了，报错并结束；
- ⑥ 否则括号配对成功，结束。

核心而简单的算术表达式中括号匹配检测程序

(为简化, 假设只有小括号)

linkStack

```
#include <iostream>
#include "linkStack.h"
using namespace std;
int main()
{   char str[20];
    linkStack<char> s;    //建立一个字符栈
    char ch;
    int i;
    cout<<"Input the string: ";
    cin.getline(str, 20, '\n');
    cout<<"str: "<<str<<endl;
```

```
    i=0;    ch=str[i++];
    while (ch!='\0')
    {   switch(ch)
        {   case '(':
            case ')':
                s.push(ch);    break;
                if (s.isEmpty())
                {   //读入一个闭括号, 栈
                    //却空, 找不到匹配的开
                    //括号
                    cout<<"An opening
                        bracket '(' is
                        expected!\n";
                    return 1;
                }
            }
```

核心而简单的算术表达式中括号匹配检测程序

(为简化, 假设只有小括号)

linkStack

```
i=0;    ch=str[i++];
while (ch!='\0')
{
    switch(ch)
    {
        case '(':    s.push(ch);    break;
        case ')':
            if (s.isEmpty())
            {
                //读入一个闭括号, 栈却空, 找不到匹配的开括号
                cout<<"An opening bracket '(' is expected!\n";
                return 1;
            }
    }
}
```

```
        else        s.pop();
        break;
    }
    ch=str[i++];
}
if (!s.isEmpty()) //式子读入结束, 发现栈中还有多余的开括号
    cout<<"A closing bracket ')' is expected!\n";
return 0;
}
```

栈的应用

- ▶ 函数调用
- ▶ 括号匹配
- ▶ 表达式计算

表达式计算

算术表达式是编程语言中一个最基本的组成元素，由操作数、运算符及括号构成。

以下分析中，为了简化，限定操作数为一位整数；运算符为加、减、乘、除四种二元运算符；括号仅含有小括号，如： $5*(7-2*3)+8/2$ 。

- ▶ 算术表达式中运算符出现在两个操作数之间，这种形式称为中缀式，运算符在前称为前缀式或波兰式，运算符在后称后缀式或逆波兰式。

中缀式有利于人的理解，但不便于计算机处理；

前缀式不便于人理解，但可去掉括号；

后缀式不便于人理解，可去掉括号，更便于计算机计算。

在编译时，编译器首先要把中缀式转换成后缀式。

表达式计算

表达式计算涉及到两个方面的工作：

- ▶ 中缀式转换为后缀式
- ▶ 后缀式计算

如，表达式 $5*(7-2*3)+8/2$ 转换为后缀式为：5 7 2 3*-*8 2/+。

手工转换时，先计算的先转换。

可以看出：

从左至右，操作数保持原来的相对位置，操作符是先计算的先出现。

后缀式经过一次从左到右的扫描即可计算出结果。

前缀式：？

后缀式的计算

如，表达式 $5*(7-2*3)+8/2$ 转换为后缀式为：5 7 2 3*- *8 2/+。

手工转换时，先计算的先转换。

在后缀式中可以看出：

从左至右，操作数保持原来的相对位置，操作符是先计算的先出现。

摆龙门阵：

计算后缀式：以 5 7 2 3*-*8 2/+ 为例

步骤	读剩的后缀式	栈中内容	步骤	读剩的后缀式	栈中内容
1	<u>5</u> <u>7</u> <u>2</u> <u>3</u> *-* <u>8</u> <u>2</u> /+		10	/+	<u>5</u> <u>8</u> <u>2</u>
2	<u>7</u> <u>2</u> <u>3</u> *-* <u>8</u> <u>2</u> /+	<u>5</u>	11	+	<u>5</u> <u>4</u>
3	<u>2</u> <u>3</u> *-* <u>8</u> <u>2</u> /+	<u>5</u> <u>7</u>	12		<u>9</u>
4	<u>3</u> *-* <u>8</u> <u>2</u> /+	<u>5</u> <u>7</u> <u>2</u>	13		
5	*-* <u>8</u> <u>2</u> /+	<u>5</u> <u>7</u> <u>2</u> <u>3</u>	14		
6	-* <u>8</u> <u>2</u> /+	<u>5</u> <u>7</u> <u>6</u>	15		
7	* <u>8</u> <u>2</u> /+	<u>5</u> <u>1</u>	16		
8	<u>8</u> <u>2</u> /+	<u>5</u>	17		
9	<u>2</u> /+	<u>5</u> <u>8</u>	18		

计算后缀式

- ① 声明一个操作数栈，依次读入后缀式中的字符。
- ② 若读到的是操作数，将其进栈；
- ③ 若读到的是运算符，将栈顶的两个操作数出栈。后弹出的操作数为被操作数，先弹出的为操作数。将出栈的两个操作数完成运算符所规定的运算后将结果进栈。
- ④ 继续读入后缀式中的字符，如上处理，最后直到后缀式中所有字符读入完毕。
- ⑤ 当完成以上操作后，栈中只剩一个操作数，弹出该操作数，它就是表达式的计算结果。

计算后缀式算法实现

calcPost

```
int calcPost(char *sufStr)
{
    int op1, op2, op;    int tmp, i;
    linkStack<int> s;
    i=0;
    while (sufStr[i]!='\0')
    {
        if ((sufStr[i]>='0')&&(sufStr[i]
        ]<='9')) //数字转为整数后进栈
        {
            tmp = sufStr[i] - '0';
            s.push(tmp);
        }
        else
        {
            op2 = s.top(); s.pop(); //栈
            顶整数出栈
            op1 = s.top(); s.pop();
```

```
            switch (sufStr[i])
            {
                case '*': op = op1*op2; break; //
                    如果是运算符为 '*', 则做*运算
                case '/': op = op1/op2; break;
                case '+': op = op1+op2; break;
                case '-': op = op1-op2; break;
            };
            s.push(op); //每一步计算结果进栈
        }
        i++;
    }
    op = s.top(); s.pop();
    return op;
}
```

从算法中可以看出，对后缀式字符串从左到右一次扫描即可计算完毕。

中缀式转后缀式

表达式 $5*(7-2*3)+8/2$ 转换为后缀式为: $5\ 7\ 2\ 3\ *\ -\ *8\ 2\ /\ +$ 。

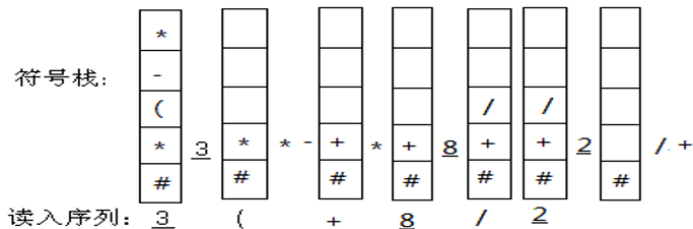
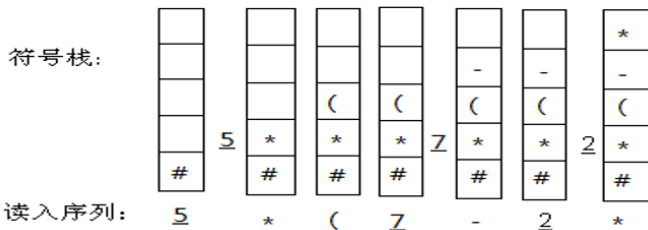
手工转换时, 先计算的先转换, 即按照计算的优先级来。

观察后缀式, 可以看出:

从左至右, 操作数保持原来的相对位置, 操作符是先计算的先出现。

中缀式转后缀式算法示例：5*(7-2*3)+8/2 转换为 5 7 2 3*-*8 2/+

设立一个用于保存运算符的堆栈，先将一个底垫' #' 压栈，设其优先级为最低。



中缀式转后缀式算法分析

- ▶ 对一个中缀表达式，从左至右顺序读入各操作数、运算符。
- ▶ 当读入的是操作数时，直接输出（如：输出到屏幕或追加到保存后缀式的字符串中）；
- ▶ 当读入的是操作符时，当读入的运算符优先级高时，因不知是否后续读入的操作符优先级更高，只能将本次读入的运算符暂存，继续读入中缀式。当读入的运算符优先级低时，才可能计算刚才暂存的运算符（即输出）；在暂存机构中，越是后面存入的操作符，优先级越高，越先出来进行计算，这种结构就是栈，处于栈顶的运算符的优先级最高。
- ▶ 相同运算符，先进栈的优先级高于后进栈的。
- ▶ 表达式中的括号也可以看作是一种操作符，其中开括号具有两面性：即将进入栈的开括号，优先级最高；已经在栈顶的开括号，优先级最低。括号在后缀式中是要消失的，消失靠闭括号。当读入一个闭括号时，计算之前进栈的运算符，直到遇到一个开括号，然后开闭括号双双消失即可。

中缀式转后缀式算法

对一个中缀表达式，从左至右顺序读入各操作数、运算符。

- ▶ 当读入的是操作数时，直接输出。
- ▶ 当读入的符号是开括号，直接进栈。
- ▶ 当读入的符号是闭括号，反复进行栈顶元素出栈、输出，直到弹栈的是开括号。
- ▶ 当读入的是操作符时，如果栈顶操作符的优先级更高，反复弹栈、输出直到栈顶元素优先级低于读入操作符的优先级，读入操作符压栈；如果栈顶操作符的优先级低，读入的运算符进栈。

中缀转后缀式算法实现

inToSufForm

```
void inToSufForm(char *inStr, char *
    sufStr)
{
    linkStack<char> s; //用字符栈
    int i,j;    char topCh;

    s.push('#'); //铺垫一个底垫
                i=0;j=0
    while (inStr[i]!='\0')
    {
        if ((inStr[i]>='0')&&(inStr[i]
            ]<='9'))
            sufStr[j++]=inStr[i]
                ++];
    }
```

```
else {
    switch (inStr[i])
    {
        case '(': s.push('('); break;
            //优先级最高, 直接入栈
        case ')': //弹栈, 弹出元素进入
            后缀式, 直到弹出一个左括号
            topCh = s.top(); s.pop();
                ;
            while (topCh!='(')
            {
                sufStr[j++] = topCh;
                topCh = s.top(); s.
                    pop();
            } // ')' 字符不入栈
            break;
    }
```

中缀转后缀式算法实现

inToSufForm

```
case '*':  
case '/':    topCh = s.top();  
             while ((topCh=='*'  
                    ')|| (topCh=='/'  
                        /'))  
                // *、/ 为左结合，  
                // 故后来者优先级低  
            {   s.pop();  
                sufStr[j++] =  
                    topCh;  
                topCh = s.top  
                    ();  
            }  
            s.push(inStr[i]);  
            break;
```

```
case '+':  
case '-':    topCh = s.top();  
             while ((topCh!='(')&&(topCh!='#'))  
                // 只有左括号和底垫优先级  
                // 比+、-低  
            {   s.pop();    sufStr[j++]  
                = topCh;    topCh = s.  
                    top();    }  
            s.push(inStr[i]);  
            break;  
        } // switch  
        i++;  
    } // else  
} // while
```

中缀转后缀式算法实现

inToSufForm

```
//将栈中还没有弹出的操作符弹空
topCh = s.top();
while (topCh != '#')
{
    sufStr[j++] = topCh;
    s.pop();
    topCh = s.top();
}
sufStr[j] = '\0'; //后缀字符串加结束符 '\0'
}
```

思考：转换表达式和计算表达式，可以合二为一吗？

- 一个容量为 25 的循环队列中，若头指针 $\text{front}=18$ ，尾指针 $\text{rear}=9$ ，则该循环队列中共有多少元素？
A. 16 B. 17 C. 9 D. 10

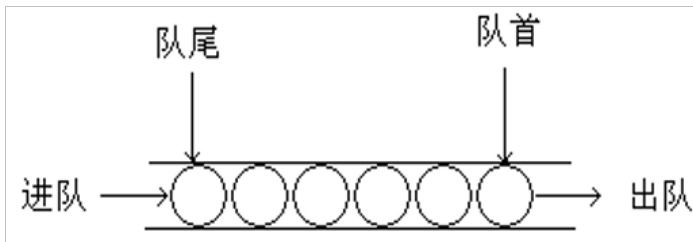
- ▶ 栈
- ▶ 顺序栈
- ▶ 链式栈
- ▶ 栈的应用
- ▶ 队列
- ▶ 顺序循环队列
- ▶ 链式队列
- ▶ 优先队列
- ▶ 队列的应用

队列的定义

- ▶ 如果元素到达线性结构的时间越早，离开的时间就越早，这种线性结构称为队 (Queue) 或者队列。
- ▶ 因为元素之间的关系是由到达、离开的时间决定的，因此队列通常被称为时间有序表。
- ▶ 而到达和离开的含义就是插入和删除操作，因此队列可以看作是插入和删除操作位置受限的线性表。

队列的定义

- ▶ 客户在银行窗口前排队存取款、计算机系统中打印管理器对打印队列的处理都采用了先来先服务 (FIFO, First In First Out) 的方式。
- ▶ 可以将队列想象为一段管道，元素从一端流入，从另一端流出。流入端称为队尾，流出端称为队首。
- ▶ 元素从队首删除的操作，称为出队 (deQueue)；元素在队尾位置插入的操作，称为进队 (enQueue)。



队列的基本操作

队列的基本操作包括：

构造队列 initialize、判队空 isEmpty、判队满 isFull、读队首元素 front、进队 enqueue、出队 dequeue、销毁队列 destroy。

队列的抽象数据类型

Data: $\{x_i | x_i \in ElemSet, i = 1, 2, 3, \dots, n, n > 0\}$ 或 Φ ; ElemSet 为元素集合。

Relation: $\{< x_i, x_{i+1} > | x_i, x_{i+1} \in ElemSet, i = 1, 2, 3, \dots, n - 1\}$, x_1 为队首, x_n 为队尾。

Operations:

initialize 输入: 无 结果: 分配相应空间及初始化。

isEmpty 输入: 无 输出: 队 Queue 为空返回 true, 否则返回 false。

isFull 输入: 无 输出: 队 Queue 为满返回 true, 否则返回 false。

front 前提: 队 Queue 非空。 输出: 返回相应队首元素的数据值, 队首元素不变。

enqueue 前提: 队 Queue 非满, 已知待进队的数据值。 结果: 将该数据值的元素队进队, 使其成为新的队尾元素。

dequeue 前提: 队 Queue 非空。 结果: 将队首元素出队, 该元素不再成为队首元素。

destroy 输入: 无 结果: 销毁并释放队 Queue 占用的空。

栈和队列

- ▶ 栈
 - ▶ 顺序栈
 - ▶ 链式栈
 - ▶ 栈的应用
- ▶ 队列
 - ▶ 顺序循环队列
 - ▶ 链式队列
 - ▶ 优先队列
 - ▶ 队列的应用

顺序队列

用一组连续的空间存储队列中的元素及元素间关系，这样存储的队列称**顺序队列**。

队列中的元素个数最多为 maxSize 个，其下标的范围从 0 到 $\text{maxSize}-1$ 。

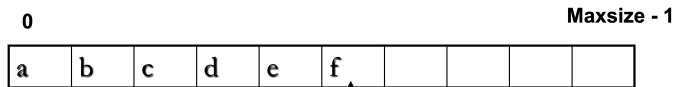
使用队首指针 Front 指示队首元素，使用队尾指针 Rear 指示队尾元素，便于出队进队操作。

顺序队列的三种组织方式

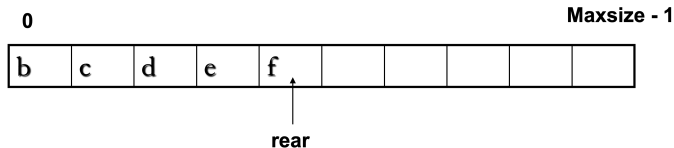
- ▶ 队头位置固定
- ▶ 队头位置不固定
- ▶ 循环队列

队头位置固定

a) 队头固定在下标 0; b) 用一个变量指出队尾位置; c) 队列为空时, 队尾位置为-1。

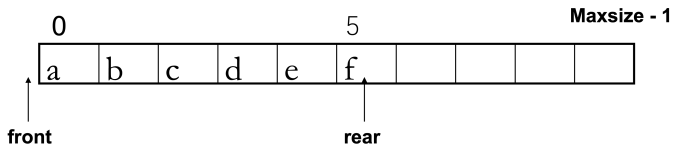


a出队

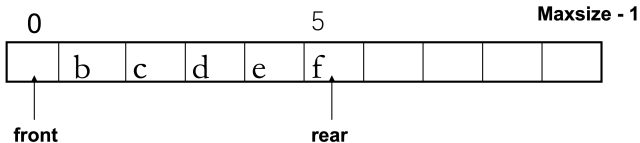


缺点: 出队会引起大量的数据移动

队头位置不固定



a出队



特点：所有操作都是 $O(1)$ ，浪费空间

顺序循环队列

队首指针 Front 给出的是实际队首元素的地址，队尾指针 Rear 给出的是实际队尾元素的下一个空间地址。

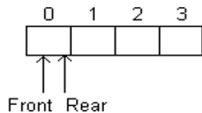
为防止队列后移造成的空间浪费，附加一个循环特征，称**顺序循环队列**。

判断队空或队满可以利用队首指针 Front 和队尾指针 Rear 的关系。

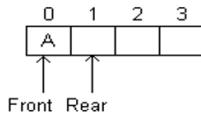
队空标志： $\text{Rear} == \text{Front}$

队满标志： $(\text{Rear} + 1) \% \text{maxSize} == \text{Front}$

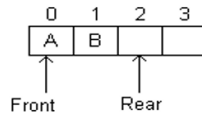
顺序循环队列



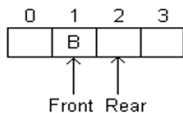
(a)初始化，队空



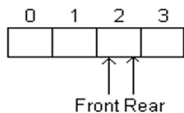
(b)A进队后



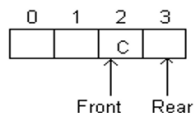
(c)B进队后



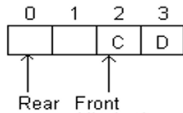
(d)A出队后



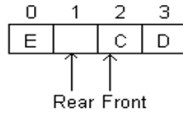
(e)B出队后，队空



(f)C进队后



(g)D进队后



(h)E进队后，队满

顺序循环队列类的定义

seqQueue

```
#ifndef SEQQUEUE_H_INCLUDED
#define SEQQUEUE_H_INCLUDED
class illegalSize{};
class outOfBound{};
template <class elemType>
class seqQueue
{    private:
    elemType *array;
    int maxSize;
    int Front, Rear;
    void doubleSpace(); //扩展队队列元素的存储空间为原来的2倍
```

```
public:
    seqQueue(int size=10); //初始化队列元素的存储空间
    bool isEmpty(); //判断队空否，空返回true，否则为false
    bool isFull(); //判断队满否，满返回true，否则为false
    elemType front(); //读取队首元素的值，队首不变
    void enqueue(const elemType &x); //将x进队，成为新的队尾
    void dequeue(); //将队首元素出队
    ~seqQueue(); //释放队列元素所占据的动态数组
};
```

顺序循环队列类成员函数的实现

seqQueue

```
template <class elemType>
seqQueue<elemType>::seqQueue(int
    size) //初始化队列元素的存储空间
{
    array = new elemType[size]; //申请实际的队列存储空间
    if (!array) throw illegalSize();
    maxSize = size;
    Front = Rear = 0;
}
```

```
template <class elemType>
bool seqQueue<elemType>::isEmpty() //判断队空否，空返回true，否则为false
{return Front == Rear;}

template <class elemType>
bool seqQueue<elemType>::isFull() //判断队满否，满返回true，否则为false
{return (Rear+1)%maxSize == Front;}

template <class elemType>
elemType seqQueue<elemType>::front() //读取队首元素的值，队首不变
{
    if (isEmpty()) throw outOfBound();    return
    array[Front]; }
}
```

顺序循环队列类成员函数的实现

enqueue 和 dequeue

```
template <class elemType>
void seqQueue<elemType>::enqueue(const elemType &x)    //将x进队，成为新的队尾
{
    if (isFull()) doubleSpace();
    array[Rear]=x;
    Rear = (Rear+1)%maxSize;
}
template <class elemType>
void seqQueue<elemType>::dequeue() //将队首元素出队
{
    if (isEmpty()) throw outOfBound();
    Front = (Front+1)%maxSize;
}
```

思考：每种操作的时间复杂度分析

栈和队列

- ▶ 栈
- ▶ 顺序栈
- ▶ 链式栈
- ▶ 栈的应用
- ▶ 队列
- ▶ 顺序循环队列
- ▶ 链式队列
- ▶ 优先队列
- ▶ 队列的应用

链式队列

用单链表存储队列中的元素及元素关系。

其中队首指针 $Front$ 指向队首结点、队尾指针 $Rear$ 指向队尾结点。

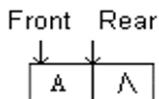
队空的条件为 $Front=Rear=NULL$ ，队满为假。



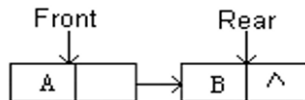
链式队列

Front=NULL
Rear=NULL

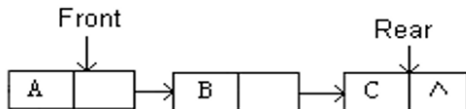
(1) 初始化 (构造) 队空



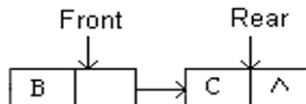
(2) A进队后



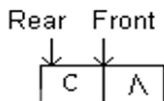
(3) B进队后



(4) C进队后



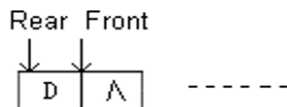
(5) A出队后



(6) B出队后

Front $\longrightarrow$ NULL
Rear $\longrightarrow$ NULL

(7) C出队后 队空



(8) D进队后

链式队列类的定义

linkQueue

```
#ifndef LINKQUEUE_H_INCLUDED
#define LINKQUEUE_H_INCLUDED
class illegalSize{};
class outOfBound{};
template <class elemType>
class linkQueue;

template <class elemType>
class Node
{
    friend class linkQueue<elemType>;
private:
    elemType data;
    Node *next;
public:
    Node(){next = NULL; }
    Node(const elemType &x, Node *p = NULL)
    { data = x; next = p;}
};
```

链式队列类的定义

linkQueue

```
template <class elemType>
class linkQueue
{ private:
    Node<elemType> *Front, *Rear;
public:
    linkQueue(){Front = Rear = NULL; }; //初始化链式队列, 使得其为空队
    bool isEmpty() {return !Front; }; //判断队空否, 空返回 true, 否则为 false
    bool isFull(){return false; }; //判断队满否, 满返回 true, 否则为 false
    elemType front(); //读取队首元素的值, 队首不变
    void enqueue(const elemType &x); //将x进队, 成为新的队尾
    void dequeue(); //将队首元素出队
    ~linkQueue(); //释放队列元素所占据的动态数组
};
```


链式队列类成员函数的实现

linkQueue

```
template <class elemType>
elemType linkQueue<elemType>::front() //读取队首
    元素的值，队首不变
{
    if (isEmpty()) throw outOfBound();
    return Front->data;
}

template <class elemType>
void linkQueue<elemType>::enqueue(const elemType
    &x) //将x进队，成为新的队尾
{
    if (!Rear)
        Front = Rear = new Node<elemType>(x);
    else
    {
        Rear->next = new Node<elemType>(x);
        Rear = Rear->next;
    }
}
```

```
template <class elemType>
void linkQueue<elemType>::deQueue() //将
    队首元素出队
{
    if (isEmpty()) throw outOfBound();

    Node<elemType> *tmp = Front;
    Front = Front->next;
    delete tmp;

    if (!Front) Rear = NULL;
}
```

链式队列类成员函数的实现

linkQueue

```
template <class elemType>
linkQueue<elemType>::~~linkQueue() //释放链式栈所占空间
{
    Node<elemType> *p;      p= Front;

    while (p)
    {
        Front=Front->next;
        delete p;      p=Front;
    }
}
#endif // LINKQUEUE_H_INCLUDED
```

思考：每种操作的时间复杂度分析

- ▶ 栈
- ▶ 顺序栈
- ▶ 链式栈
- ▶ 栈的应用
- ▶ 队列
- ▶ 顺序循环队列
- ▶ 链式队列
- ▶ 优先队列
- ▶ 队列的应用

优先队列

- ▶ 有时进入队列中的元素具有优先级 (优先级可用一个优先数表示, 一般优先数越小优先级越高), 出队时是按照优先级越高的元素出队越早, 优先级越低出队越晚, 优先级相同者按先进先出的原则处理, 这种队列称**优先队列**。
- ▶ 优先队列存储方式: 顺序存储、链式存储。

顺序优先队列

顺序优先队列中，用数组存放元素。

有两种存放策略：

- 1) 进队按时间顺序存放，出队是优先级高者出队。
- 2) 进队按优先级顺序存放，出队是队首出队。

顺序优先队列

第一种策略：

- ▶ 进队时，按照下标由小到大的顺序即直接将元素放队尾，时间为 $O(1)$ 。
- ▶ 出队时，从所有元素中找到优先级最高的元素，然后删除。
- ▶ 为避免整个队列的后移，造成空间的浪费，当有元素出队时将队列中最后一个元素移到出队元素所在的存储位置，这样队列始终从 0 下标开始到某个下标终止，中间不会出现空隙。故不需要像普通队列顺序存储时使用循环技术。查找最高优先级元素时间为 $O(n)$
- ▶ 设队尾 Rear 指针指向实际队尾元素的后一单元。
则队空的条件为：Rear = 0； 队满的条件为：Rear = maxSize。

顺序优先队列

第二种策略：

- ▶ 元素按照优先数由小到大排列。
- ▶ 元素进队时，先在队列中找到合适的插入位置，移动后面的元素，将新进元素插入，时间复杂度为 $O(n)$ 。
- ▶ 出队时，删除队首即 0 下标元素即可，为了避免后面元素的移动，可以采用顺序循环队列，时间复杂度为 $O(1)$ 。
- ▶ 设队尾 `Rear` 指针指向实际队尾元素的后一单元。
则队空的条件为：`Rear == Front`;
队满的条件为：`(Rear+1) % maxSize == Front`。

第二种策略：

- ▶ 单链表中的结点按照元素优先级递减（即优先数增大）的次序排成一个有序链表，元素优先级最高的结点就是首结点。
- ▶ 元素出队即删除首结点，时间复杂度为 $O(1)$ ；
- ▶ 元素进队，需要按照优先级大小找到合适的插入位置，然后插入元素结点，时间复杂度为 $O(n)$ ，但不会引起队列中原有元素在内存中移动。

栈和队列

- ▶ 栈
- ▶ 顺序栈
- ▶ 链式栈
- ▶ 栈的应用
- ▶ 队列
- ▶ 顺序循环队列
- ▶ 链式队列
- ▶ 优先队列
- ▶ 队列的应用

队列的应用：单服务窗口模拟

- ▶ 提供的服务窗口只有一个。
- ▶ 可以采用队列对单服务窗口进行模拟，完成诸如计算用户的平均等待时间等任务。其结果可以用来了解单服务窗口的情况，从而辅助对相关问题进行决策。

单服务窗口模拟

假设有一个小的银行服务网点，每天早晨 9 点开门，下午 4 点 30 关门。一般开门后 10 分钟内就会来一个客户，之后在上一个客户来后 5 分钟内就会来下一个客户。

客户办理的业务分三种：取款、存款、办理网银，它们分别耗时 2 分钟、4 分钟和 10 分钟。

试模拟这个服务网点的服务，并计算出每个客户的平均等待时间。

单服务窗口模拟

people

设置客户属性：

```
struct timeT
{ int hour; int minute; };
struct people
{
    timeT arrivePoint;    //客户到达银行时间
    timeT serveBegin;     //客户开始被服务时间
    int serveType;        //业务种类  0 存款  1取款  2办网银
};
```

单服务窗口模拟

入队操作：

第一个客户到达，到达时间为 9 点 + $m(0-10)$ 分（随机），业务种类随机，被服务时间为该客户到达时间。描述完毕，第一个客户进队。

第二个客户到达，到达时间为第一个客户达到时间 + $m(0-5)$ 分之间的随机值，业务种类随机，被服务时间为该客户到达时间和上一个客户结束服务时间的最大值。描述完毕，第二个客户进队。

第三个客户，计算到达时间、服务种类、开始时间，进队。

——

直到某个客户到达时间超过下午 4:30，此时结束。试模拟这个服务网点的服务，并计算出每个客户的平均等待时间。

单服务窗口模拟

出队操作：

反复出队，累计每个人的等待时间（服务开始时间-到达时间），直到队空。

计算每人的平均等待时间。

实现由同学课后自行完成。

多窗口服务：

需要两个队列配合。在等待区等候的客户形成一个普通队列（先来先服务，即先进先出）。

在各个窗口被服务的客户形成一个优先队列（先来的未必先出，先结束的先出）

小结

- ▶ 栈虽然只是在线性表操作基础上限制了插入、删除的位置，使得插入、删除操作只能在表的同一个端点进行，可以看作是一种操作受限的线性表。
- ▶ 栈在计算机系统中是一种非常重要的数据结构。除了介绍的符号匹配、表达式计算，系统在函数调用、递归中都是以栈结构为基础。
- ▶ 栈有着非常独特的一组常见操作：进栈、出栈、求栈顶元素、判栈空、判栈满等。
- ▶ 在物理实现上虽然可以有顺序和链式两种存储方式，鉴于其操作都在一端进行，顺序存储是栈最常使用的存储方式。
- ▶ 队列可看作是限制了插入、删除操作位置的另外一种线性表，元素的插入、删除分别在表的两端进行。

小结

- ▶ 除了日常生活中看到的实际队列可以用它来实现，计算机操作系统中许多对象的管理都利用了队列，如打印队列、进程队列等，因此它也是一种重要的数据结构。
- ▶ 队列的常见操作对照生活经验是最容易想到的：进队、出队、求队首元素、判空、判满等。
- ▶ 队列的顺序循环存储和链式存储的时间复杂度都是 $O(1)$ ，考虑到链式存储每个结点额外的空间开销，顺序循环队列是最常用的存储结构。
- ▶ 优先队列不再按照先进先出的原则，优先级高者先出队。无论顺序还是链式优先队列，其进队、出队操作的时间复杂度为 $O(1)$ 、 $O(n)$ 或者 $O(n)$ 、 $O(1)$ 。
- ▶ 后面章节讨论过二叉树后，用二叉树作为工具以堆的方式存储优先队列时，时间复杂度能降到 $O(\log_2 n)$ 。